

Day04 Part B-02 学习文档 v1.0 : Runtime 如何思考 (How Runtime Thinks) - Runtime State Lifecycle

本文是《从零实现 Agent Runtime》学习阶段的 Day04 Part B-02 正式学习文档。

Part B-01 解决的是 Runtime State 是什么；Part B-02 继续追问：Runtime State 是什么时候创建、如何演化、何时持久化、如何销毁，以及崩溃后如何恢复继续执行。

与上一节的联系

Part B-01 的核心结论是：

Runtime State 是 Runtime 对当前世界的认知，Agent Loop 的每一步都在推动 Runtime State 从一个状态迁移到下一个状态。

但是上一节还留下了一个更工程化的问题：

这些 State 是什么时候创建的？什么时候修改？什么时候销毁？什么时候持久化？

Part B-02 的核心结论是：

Runtime 不只是维护一个 State，而是在编排一个不断创建、恢复、演化、持久化、销毁，并且可以在崩溃后恢复继续执行的 State 生命周期。

可以这样串起来：

Part A:
Prompt 不是 Context

Part B-01:
Context 来自 Runtime State 的 Projection

Part B-02:
Runtime State 本身有完整生命周期

Part C:
Context Builder 如何把 Runtime State 投影成 Provider Messages

目录

为什么必须讲 Runtime State Lifecycle

Runtime State 是什么时候诞生的

Runtime State 不是从零开始，而是 Create + Restore

Runtime State 如何在 Agent Loop 中演化

Agent Loop 本质上是 State Transition Loop

Runtime State 的生命周期分类

Ephemeral State 与 Persistent State

Persistence Policy：不同 State 的持久化策略不同

为什么 Context 和 Prompt 通常不持久化

Runtime 如何正常结束

Crash Recovery 与 Durable Execution

Checkpoint 保存的是 Snapshot，不是 Action Replay

Recoverable World State

Task、Session、Execution 的边界修正

Runtime State Orchestration：状态如何协同

Event Bus、Policy 与 State Owner

本地持久化、隐私与重建成本

统一 Runtime 世界观

Part B-02 核心结论

下一节学习计划

写书 TODO

写书素材

本 Part 核心认知升级

本章思考题

1. 为什么必须讲 Runtime State Lifecycle

很多人第一次写 Agent Runtime，会写出这样的代码：

```
const state = {};  
  
while (true) {  
  const response = await llm(...);  
  updateState(response);  
}
```

这个写法会给人一种错觉：

State 一旦创建，就一直存在。

真正工业 Runtime 不是这样。它更像：

```
Runtime Session 创建  
  |  
  v  
Runtime 创建  
  |  
  v  
Runtime State 创建  
  |  
  v  
恢复 Persistent State  
  |  
  v  
Agent Loop 中不断演化  
  |  
  v  
按策略部分持久化  
  |  
  v
```

Runtime 结束或崩溃
|
v
清理、销毁或恢复继续执行

所以 Part B-02 关注的是：

State 从哪里来
State 如何变化
State 哪些会留下
State 哪些会消失
State 崩溃后如何恢复

如果这些问题回答不出来，Runtime 就还没有真正完成设计。

2. Runtime State 是什么时候诞生的

Runtime State 不是在程序启动时诞生的。

例如 Cursor App 已经打开，并不代表某个 Agent Runtime State 已经存在。ChatGPT 页面已经打开，也不代表一次 Agent Loop 的 Runtime State 已经开始运行。

Runtime State 更准确的出生时机是：

一次 Runtime Session 或一次 Execution 开始时。

抽象关系可以先画成：

Application
|
v
Runtime Session
|
v
Runtime
|
v
Runtime State

需要注意：

Application 生命周期 != Runtime 生命周期
Conversation 生命周期 != Runtime 生命周期

例如同一个 Conversation 中，上午用户问：

帮我解释 React Fiber

下午用户又问：

帮我修复 login bug

Conversation 可能还是同一个，但 Agent 的执行过程已经是新的 Runtime Session 或新的 Execution。

3. Runtime State 不是从零开始，而是 Create + Restore

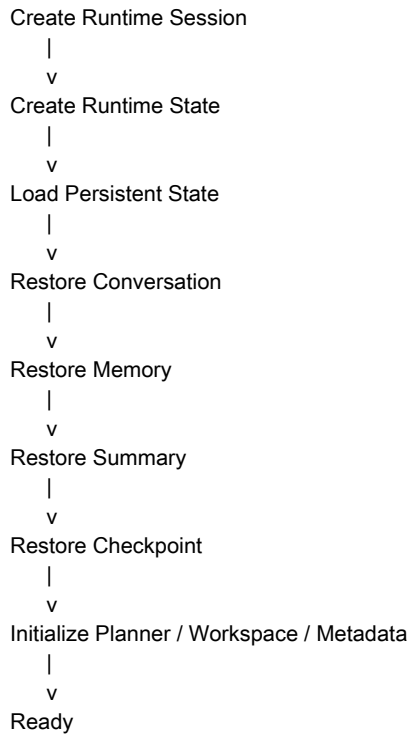
很多人会以为：

new RuntimeState()

就等于：

{}

工业 Runtime 不是从纯空对象开始。更准确的流程是：

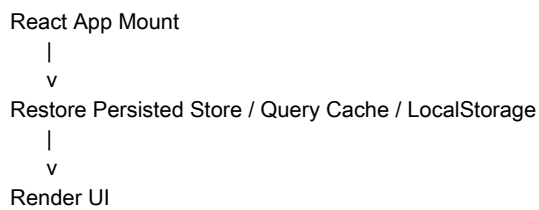


所以 Runtime 的第一步不是调用 LLM，而是恢复世界。

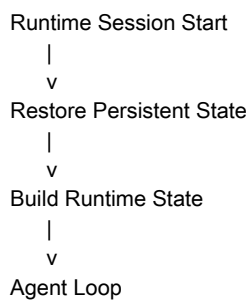
可以总结为：

Runtime Start = Create + Restore

这个过程和 React 很像：

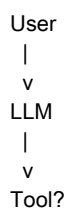


Agent Runtime 也是：



4. Runtime State 如何在 Agent Loop 中演化

Day02 中我们看到的 Agent Loop 看起来像：



```
|
v
LLM
|
v
Final Answer
```

但 Part B-02 要从 State 的角度重新理解它：

```
State S0
|
v
Action
|
v
State S1
|
v
Action
|
v
State S2
```

LLM 和 Tool 都只是推动 State 迁移的 Action。

以用户请求为例：

帮我修复 login.ts 的所有 TypeScript 错误。

Runtime 刚开始时可能是：

```
Conversation:
  User: 修复 login.ts

Workspace:
  Empty

Planner:
  null

CurrentStep:
  Idle

Diagnostics:
  null

CurrentTool:
  null
```

LLM 生成计划后，真正变化的是：

```
PlannerState:
  null
  |
  v
  Plan(ReadFile -> Analyze -> Modify -> TypeCheck)
```

Runtime 调用 Tool 时，真正变化的是：

```
CurrentTool:
  null -> ReadFile

CurrentStep:
  Planning -> ToolCalling
```

Tool 返回文件后，真正变化的是：

```
WorkspaceState:
  CurrentFile = login.ts
```

Content = ...

DiagnosticsState:
5 Errors

然后 Context Builder 基于新的 Runtime State 重新构建 Context :

```
Conversation + Planner + Workspace + Diagnostics
|
v
Context Builder
|
v
Provider Messages
```

5. Agent Loop 本质上是 State Transition Loop

完整过程可以这样画：

```
S0
|
v
LLM
|
v
S1
|
v
Tool
|
v
S2
|
v
Context Builder
|
v
S3
|
v
LLM
|
v
S4
|
v
Tool
|
v
S5
|
v
Done
```

因此 Runtime 被称为 State Machine，不是因为它只有一个简单的 status 字段，而是因为每一个动作都会造成状态迁移：

```
User Message    -> ConversationState 更新
LLM Response    -> PlannerState 更新
Tool Result     -> WorkspaceState 更新
Permission Result -> WorkflowState 更新
Summary Trigger  -> SummaryState 更新
Memory Extract   -> MemoryState 更新
```

本章核心句：

Agent Loop 的每一次循环，本质上都不是为了再次调用 LLM，而是在推动 Runtime State 从当前世界演化到下一个世界。

6. Runtime State 的生命周期分类

Runtime State 不是一个整体。它里面有许多生命周期不同的 State。

很多新人会想：

```
const runtimeState = {  
  conversation,  
  memory,  
  workspace,  
  planner,  
  context,  
  tool,  
  summary,  
};
```

然后问：

是不是整个 runtimeState 都要保存到数据库？

答案是：不是。

真正工业 Runtime 必须先区分：

- 哪些 State 是短命的
- 哪些 State 是长命的
- 哪些 State 可以重建
- 哪些 State 不可重建
- 哪些 State 必须用于恢复执行

7. Ephemeral State 与 Persistent State

Ephemeral State 是只对当前一次运行、当前一步、当前一轮 LLM 或当前 Tool 调用有意义的状态。

典型例子：

- Current Tool
- Current Context
- Streaming Buffer
- Retry Count
- Current Provider
- Current Cost
- Current Latency
- Current Token Usage
- Tool Arguments

这些状态的生命周期可能只有几十毫秒、几秒或几十秒。Runtime 结束后，它们通常直接释放。

Persistent State 是需要跨 Runtime、跨 Session、跨进程，甚至跨天继续存在的状态。

典型例子：

- Conversation
- Memory
- Summary
- Workflow Snapshot
- Checkpoint

可以这样概括：

- Runtime State

```

|
+-- Ephemeral
| |
| +-- Current Tool
| +-- Current Context
| +-- Retry Count
| +-- Streaming Buffer
| +-- Current Provider
| +-- Token Usage
|
+-- Persistent
  |
  +-- Conversation
  +-- Memory
  +-- Summary
  +-- Workflow Snapshot
  +-- Checkpoint

```

需要特别注意：一个领域对象内部也可能既有 Ephemeral 字段，也有 Persistent 字段。

例如 Workspace：

```

CurrentFile    -> Ephemeral
OpenFiles      -> 多数情况下 Ephemeral
Git Diff       -> 可重建，视情况保存
Code Index     -> 重建成本高，通常缓存
Workspace Snapshot -> 恢复需要时保存

```

Planner 也是如此：

```

CurrentStep    -> 多数情况下 Ephemeral
WaitingApproval -> 需要 Checkpoint
Workflow Position -> Durable Execution 中需要保存

```

所以判断标准不是对象名字，而是这个字段对恢复世界是否有价值。

8. Persistence Policy：不同 State 的持久化策略不同

Persistent State 也不是在同一个时间点统一保存。

真正工业 Runtime 的思想是：

每一种 State 都有自己的 Persistence Policy。

Conversation：Append Immediately

Conversation 是 Event Log，事件发生后应尽快追加。

```

User Message    -> append
Assistant Message -> append
Tool Call       -> append
Tool Result     -> append

```

Conversation 的策略通常是：

Append-only

原因是事件日志不能丢。

Summary：Threshold Driven

Summary 不能每条消息都更新，否则会频繁调用 LLM。

常见触发条件：

Message Count > 200

Token Count > 30000
Context Budget Pressure

Summary 的策略通常是：

Threshold Driven

Memory：Semantic Driven

Memory 不是每句话都保存，而是判断是否有长期价值。

例如：

以后回答我都用中文。

这类信息值得保存。

但：

今天北京下雨。

不一定值得作为长期 Memory。

Memory 的策略通常是：

Semantic Driven

Checkpoint：Milestone Driven

Checkpoint 不是每一步都保存，而是在可恢复的稳定节点保存。

常见触发条件：

- 完成一个 Plan Step
- 进入 Waiting Approval
- Workflow Pause
- Human Approval 前后
- Tool 最终成功
- Tool 最终失败
- User Interrupt

Checkpoint 的策略通常是：

Milestone Driven

Workspace Cache：Usually Not Persist 或 Cache Persist

Workspace 很多信息可以重新扫描：

- Git Diff
- Open Files
- Diagnostics
- Symbols

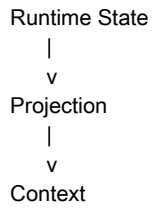
但大型 Code Index、Embedding Index 等重建成本高的数据，可能需要本地缓存。

策略表

| State | Persistence Policy | 原因 || --- | --- | --- || Conversation | Append Immediately | 事件日志不能丢 || Summary | Threshold Driven | 节省 Token 和 LLM 成本 || Memory | Semantic Driven | 只保存有长期价值的信息 || Checkpoint | Milestone Driven | 支持恢复执行 || Workspace Cache | Usually Not Persist / Cache Persist | 取决于重建成本 || Current Context | Never Persist | 每轮重新构建 || Current Tool | Never Persist | 生命周期太短 || Retry Count | Never Persist | 当前执行状态 |

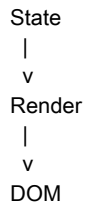
9. 为什么 Context 和 Prompt 通常不持久化

Day04 Part A 已经建立了一个核心模型：



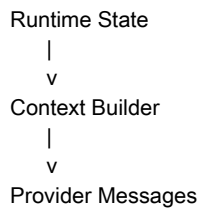
Context 本质上是 Projection，不是 Source of Truth。

React 类比：



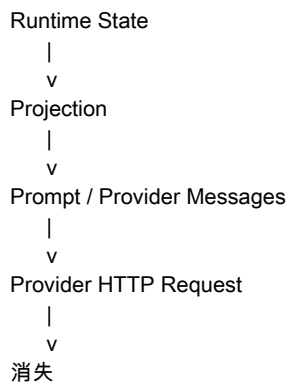
DOM 通常不保存，刷新后重新 Render。

Agent 也是：



恢复时恢复的是 Runtime State，然后重新 Build Context。

Prompt 更短命。它只是 Provider Request 中临时生成的 Payload：



所以：

Conversation 可以保存
Memory 可以保存
Summary 可以保存
Checkpoint 可以保存
Context 通常不保存
Prompt 通常不保存

10. Runtime 如何正常结束

Runtime 正常结束时，不是简单地 destroy()。

真正流程更像：

```
Done
|
v
Flush State
|
v
Extract Memory if needed
|
v
Update Summary if needed
|
v
Save Metrics / Cost / Trace
|
v
Release Resource
|
v
Destroy Runtime
```

结束后会释放：

```
Current Tool
Current Context
Streaming Buffer
Planner Cache
Workspace Cache
RuntimeState in Memory
```

留下：

```
Conversation
Memory
Summary
Checkpoint
Trace / Metrics
```

所以 Runtime

结束并不意味着所有东西都消失，而是意味着内存中的运行现场被清理，只留下按策略持久化的部分。

11. Crash Recovery 与 Durable Execution

工业级 Runtime 必须处理非正常结束。

例如：

```
Agent 已运行 20 分钟
读取 200 个文件
修改 35 个文件
测试执行到第 18 分钟
服务器突然 Crash
```

如果没有恢复机制，用户只能从头再来。

Checkpoint 的作用是：

Runtime 在运行过程中主动保存可恢复的存档点。

例如当前 Plan：

```
1. Read Project  ✓
2. Analyze      ✓
3. Modify       ✓
4. Test        running
5. Commit      pending
```

Checkpoint 保存的可能是：

- Conversation
- PlannerState
- Current Step
- Workflow Variables
- Memory
- Workspace Snapshot
- Diagnostics

Crash 后：

```
Load Checkpoint
  |
  v
Restore Runtime State
  |
  v
Context Builder 重新生成 Context
  |
  v
Resume Execution
```

Durable Execution 可以用一句话解释：

Runtime 的执行过程不会因为进程崩溃而丢失。

它恢复的不是 LLM，不是 Prompt，而是 Execution State。

LangGraph、Temporal、Durable Functions 这类系统复杂，很大一部分原因就是维护：

- 当前执行到哪个 Node
- 当前变量是什么
- 哪些 Step 已完成
- 哪些副作用已发生
- 哪些 Step 可以重试
- 哪些状态可以作为恢复点

12. Checkpoint 保存的是 Snapshot，不是 Action Replay

有一个常见误解：

恢复 Execution，是不是要保存并回放所有动作？

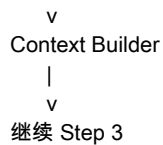
一般不是。

Agent Runtime 更常见的方式是保存 Snapshot：

```
const checkpoint = {
  planner: {
    currentStep: 3,
    currentNode: "Modify",
    status: "running",
  },
  conversation: [...],
  memory: [...],
  workspace: {...},
  variables: {...},
};
```

恢复时：

```
Load Snapshot
  |
  v
Runtime State S3
  |
```



这不是 Action Replay。

Event Sourcing 是另一种架构，它会保存事件：

ReadFile
Analyze
WriteFile
RunTest

然后通过 Replay 恢复 State。

但 Agent Runtime 很少完全依赖 Replay，因为 LLM 调用不是确定性的：

同一个 Prompt 第一次可能回答 A
第二次可能回答 B

因此工业 Runtime 更偏向：

Checkpoint Snapshot > Action Replay

13. Recoverable World State

Checkpoint 保存的不是 CPU 寄存器，也不是线程执行到第几行代码。

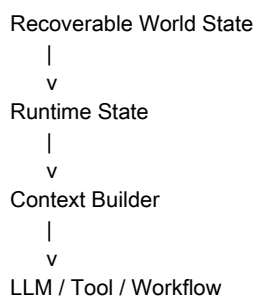
它保存的是：

可恢复的业务现场，或者说 Recoverable World State。

它包括：

Conversation：聊到哪里了
Planner：执行到哪一步了
Workspace：当前世界是什么样
Memory：长期记忆是什么
Variables：当前变量值
Diagnostics：当前错误是什么
Approval：是否正在等待用户批准

恢复时，Runtime 根据这些状态重新启动思考：



所以更准确地说：

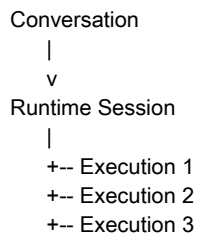
Agent 恢复的不是代码执行栈，而是 Agent 对世界的认知现场。

14. Task、Session、Execution 的边界修正

学习过程中有一个重要修正：

“一次任务对应一次 Runtime”适合做概念模型，但工业实现不一定这么绝对。

更精确的层次可以是：



每次用户发新消息，可能创建新的 Execution，而不是销毁并重建整个 Runtime。

判断什么时候创建 Execution 通常不需要 LLM。它是 Runtime 生命周期逻辑：



一个大任务内部拆出来的子任务，通常是 PlannerState 的一部分：

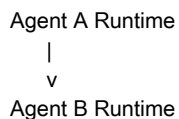
Goal:
修复整个项目

Plan:

1. Read Project
2. Analyze
3. Modify
4. Test
5. Commit

这并不意味着多个 Runtime。

多个 Runtime 通常出现在 Multi-Agent 或子 Agent 场景：



15. Runtime State Orchestration：状态如何协同

Part B-02 后半段最重要的补充是：

真正复杂的不是 State 本身，而是 State 的编排。

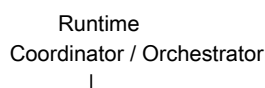
例如 Conversation 更新后，Summary 是否需要更新？Memory 是否需要抽取？Checkpoint 是否需要保存？

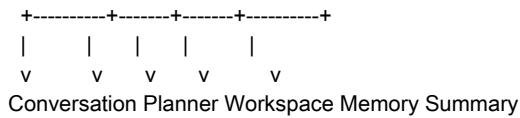
不应该设计成：

```
ConversationManager 直接调用 SummaryManager
ConversationManager 直接调用 MemoryManager
ConversationManager 直接调用 CheckpointManager
```

这样会造成模块互相引用，最后形成网状依赖。

更好的模型是：





所有 Manager 只管理自己的 State。

```
ConversationManager -> state.conversation
SummaryManager     -> state.summary
MemoryManager      -> state.memory
PlannerManager     -> state.planner
```

它们不互相直接调用。

Runtime Coordinator 负责生命周期和编排。

16. Event Bus、Policy 与 State Owner

真正工业 Runtime 往往是：

Mediator + Event Bus + Policy + State Owner

Event Bus

Conversation 新增消息后，不直接调用 Summary，而是发事件：

```
eventBus.emit("conversation.updated", {
  conversationId,
  messageId,
});
```

关心这个事件的模块自己订阅：

```
eventBus.on("conversation.updated", () => {
  summaryPolicy.check();
});

eventBus.on("conversation.updated", () => {
  memoryExtractor.run();
});

eventBus.on("conversation.updated", () => {
  metrics.increment("message.count");
});
```

这样 Conversation 不知道 Summary、Memory、Metrics 的存在。

Policy

Policy 决定是否行动：

```
SummaryPolicy.shouldSummarize()
MemoryPolicy.shouldRemember()
CheckpointPolicy.shouldSave()
ContextPolicy.shouldInclude()
```

这比把所有判断写进一个巨大的 if else 更容易维护。

State Owner

每种 State 应该有唯一 Owner。

```
ConversationState -> ConversationManager
MemoryState       -> MemoryManager
PlannerState      -> PlannerManager
SummaryState      -> SummaryManager
```

其他模块可以读取，但不能直接改。

否则会出现：

Summary 修改 Conversation
Memory 修改 Conversation
Planner 修改 Conversation
Tool 修改 Conversation

最后无法判断是谁造成了状态污染。

所以工业 Runtime 要解决的不只是状态保存，而是状态治理：

谁能修改
谁能订阅
谁能触发
谁能持久化
谁能恢复

17. 本地持久化、隐私与重建成本

以 Cursor 这类本地代码 Agent 为例，同一个账号下不同设备或工作区的 Conversation 可能是隔离的。

这背后通常有几个原因：

隐私：代码和上下文不适合全部上传
速度：本地 SQLite 或本地索引比云端更快
工作区相关性：Git、Index、OpenFiles 与本地目录绑定
成本：很多 Workspace 状态可以重新扫描

但本地保存不等于没有持久化。

可能本地持久化：

Conversation
Metadata
部分 Checkpoint
Code Index Cache
Workspace Cache

是否保存的核心权衡不是“能不能存”，而是：

Reconstruction Cost
Storage Cost
Latency
Privacy
Consistency

可以总结成表：

数据 重建成本 是否保存	--- --- ---	Current Context 极低 不保存	Current Tool 极低 不保存
Git Diff 低 视情况	Diagnostics 中等 视情况	Code Index 高 通常缓存	Conversation 不可重建
保存	Memory 不可重建或重建成本极高	保存	Summary 可重建但消耗 Token 通常保存
Checkpoint	恢复执行必需 关键节点保存		

18. 统一 Runtime 世界观

从 Day01 到 Day04，认知可以这样升级：

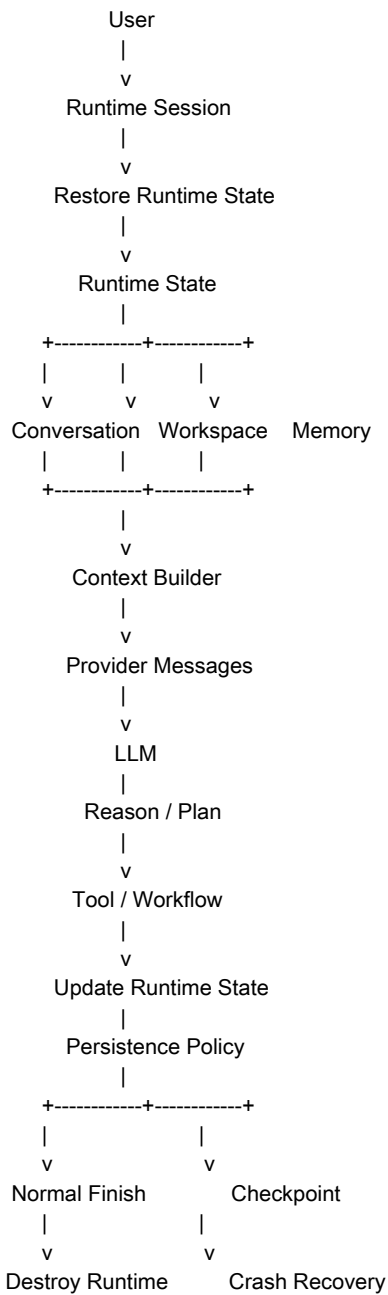
Day01:
Agent = LLM + Tool

Day02:
Agent = Runtime + LLM + Tool + Loop

Day03:
Runtime = Coordinator + Planner + Tool Registry + Memory + Provider

Day04:
Runtime = Runtime State + Context Builder + Policy + Orchestration

最终图：



这里的核心不再是 Prompt，而是：

Runtime State
Context Builder
Persistence Policy
Checkpoint
Recoverability
State Orchestration

19. Part B-02 核心结论

Part B-02 的核心结论可以分成五层。

第一层：

Runtime State 不是随着程序启动诞生，而是随着 Runtime Session 或 Execution 开始而创建。

第二层：

Runtime State 不是从零开始，而是 Create + Restore。

第三层：

Agent Loop 本质上是推动 Runtime State 不断迁移的 State Transition Loop。

第四层：

不同 State 的生命周期和 Persistence Policy 完全不同，工业 Runtime 不会统一保存整个 Runtime State。

第五层：

工业 Runtime 真正复杂的地方，是让 State 可以被正确编排、治理、持久化、恢复，并在崩溃后继续执行。

最终一句话：

Runtime 的本质不是调用 LLM，而是维护 Runtime State，并保证 Runtime State 可以正确演化、持久化、恢复和继续执行。

20. 下一节学习计划

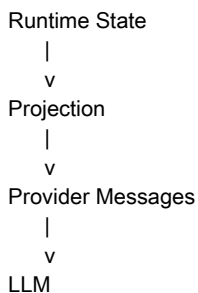
下一节进入：

Day04 Part C：Context Builder

学习目标：

- 为什么 Runtime State 不能直接发给 LLM；
- Projection 到底是什么；
- Context Builder 的职责边界；
- Provider Messages 如何生成；
- Context Builder 如何选择 Conversation、Summary、Memory、Workspace；
- Token Budget 如何影响 Context；
- Context Compression 与 Message Selection；
- Prompt Assembly 与 Provider Adapter 的关系；
- 为什么 Context Builder 是 Runtime 最核心的模块之一。

核心认知路径：



21. 写书 TODO

- 新增章节：《Runtime State Lifecycle》
- 新增章节：《Runtime State Orchestration》
- 补充 Runtime Session、Execution、Task 的边界说明
- 增加 Runtime State 生命周期图
- 增加 Ephemeral State vs Persistent State 表格
- 增加 Persistence Policy 表格
- 增加 Checkpoint Snapshot 示例
- 增加 Recoverable World State 小节
- 增加 Context 不持久化、恢复后重新生成 Context 的 React 类比
- 增加 State Ownership 与 State Governance 小节
- 将 Runtime = State Machine 升级为 Runtime = State Orchestrator

22. 写书素材

素材一：React 类比

React:
State -> Render -> DOM

Agent:
Runtime State -> Context Builder -> Context / Provider Messages

素材二：Checkpoint 不是 Action Replay

Checkpoint 保存 Snapshot，不保存所有动作。
恢复时恢复 Runtime State，再重新生成 Context。

素材三：Recoverable World State

Agent 恢复的不是 CPU 执行栈，而是 Agent 对世界的认知现场。

素材四：Runtime 像操作系统

OS 维护 Process / Memory / File / Permission
Runtime 维护 Conversation / Memory / Workspace / Planner / Permission

素材五：Tool 的重新定义

Tool 的价值不在于返回数据，而在于改变 Runtime 对世界的认知。

素材六：Runtime Engineering 的方向

Prompt 是 Projection 的结果，不是 Runtime 的起点。

23. 本 Part 核心认知升级

最开始的认知：

Runtime 调 LLM
LLM 调 Tool
Tool 返回结果

升级后的认知：

Runtime 维护 Runtime State
LLM 和 Tool 都只是推动 State Transition 的 Action
Context Builder 把 Runtime State 投影成 LLM 可见的世界
Persistence Policy 决定哪些 State 留下
Checkpoint 让 Runtime 可以在崩溃后恢复继续执行
Orchestrator / Event Bus / Policy / Owner 共同治理 Runtime State

最终认知：

Runtime 是 AI Agent 的状态编排器。它维护世界、调度推理、执行工具、管理持久化，并保证 Agent 在真实世界中可以连续、可靠地运行。

24. 本章思考题

- 为什么 Conversation 应该 Append Immediately，而 Summary 不应该每条消息都更新？
- 为什么恢复 Execution 不是恢复 Context，而是恢复 Runtime State 后重新生成 Context？
- Checkpoint 保存 Snapshot 相比 Action Replay，有什么优势和代价？
- 如果所有模块都能直接修改 runtimeState，会带来哪些调试和一致性问题？
- Event Bus 如何降低 Conversation、Summary、Memory、Metrics 之间的耦合？
- Workspace 中哪些数据应该持久化，哪些应该重新扫描？判断标准是什么？
- 为什么“错误状态”有时也应该 Checkpoint？它需要满足什么条件？
- Runtime Session、Execution、Task 三者的边界如何划分？
- 为什么 Prompt 不应该被看成 Runtime 的核心资产？
- 如果要从零实现一个最小 Agent Runtime，哪些 State Manager 必须先出现？

来源

- ChatGPT 分享学习记录：<https://chatgpt.com/share/6a5dd64e-390c-83ee-b4cc-9b5a4e5f0eab>
- 本地源记录：source/day04-part-b-02-chatgpt-share-source.md